

MODUL VII

Testing, Debugging dan Security

Tujuan

- ◆ Mahasiswa dapat menulis dan menjalankan Feature Test untuk endpoint CRUD.
- ◆ Mahasiswa dapat menggunakan Log facade untuk mencatat operasi CRUD.
- ◆ Mahasiswa dapat mengkonfigurasi rate limiting pada route API.
- ◆ Mahasiswa dapat menjalankan **composer audit** dan mengidentifikasi/vulnerabilitas.
- ◆ Mahasiswa memahami dasar debugging dengan dd() dan Log.
- ◆ skenario bahwa hanya admin dapat melakukan delete.

Materi

1. Testing Dasar (PHPUnit & Laravel)
 - 1.1. Lokasi tests/Feature dan tests/Unit
 - 1.2. Feature Test: simulasi HTTP request ke endpoint
 - Artisan: `php artisan make:test ItemApiTest --unit=false`
 - Assertions:

```
this->getJson('/api/v1/items')
    ->assertStatus(200)
    ->assertJsonStructure(['status','data'])

this->postJson(...)
    ->assertStatus(201)
    ->assertJsonFragment([...])

$this->deleteJson(...)->assertStatus(403)
```
 - 1.3. Menjalankan test: `php artisan test` atau `vendor/bin/phpunit`
2. Debugging & Logging
 - 2.1. Debugging:
 - `dd()`, `dump()`, `ray()` (pakai spatie/laravel-ray)
 - Xdebug + IDE integration
 - 2.2. Logging:
 - Facade Log (`Log::info`, `Log::error`)
 - Channel: `daily`, `single`, `stack` (`config/logging.php`)
 - Penulisan log di Service Layer untuk setiap aksi penting
3. Keamanan Lanjutan
 - 3.1. CSRF
 - API routes (**`routes/api.php`**) tidak memerlukan CSRF token

- Web routes (**routes/web.php**) dilindungi oleh **VerifyCsrfToken** middleware
- 3.2. Rate Limiting
- Middleware **throttle:rate,minutes**
 - Default di **app/Http/Kernel.php** pada group 'api':
'throttle:api' (default 60 req/1 mnt)
 - Custom: **Route::middleware('throttle:100,1')->group(...)**
- 3.3. Audit Dependency
- **composer audit** (scan vulnerability)
 - **composer audit --format=json**
 - Update package:
composer update vendor/package --with-dependencies

Praktikum

1. Pastikan Anda switch dari branch main ke branch feature/testing-debug-security:

```
cd C:\laragon\www\inventory
git checkout main
git pull origin main
git checkout -b feature/testing-debug-security
```

2. Tambah Logging di Service Layer

- 2.1. Buka app/Services/ItemService.php, tambahkan
use Illuminate\Support\Facades\Log;

- 2.2. Contoh untuk method create:

```
public function create(array $data) {
    $item = Item::create($data);
    Log::info('Item created', [
        'id'=>$item->id,
        'data'=>$data
    ]);
    return $item;
}
```

- 2.3. Lakukan serupa di update() dan delete():

```
Log::info('Item updated', ['id'=>$id,'changes'=>$data]);
Log::info('Item deleted', ['id'=>$id]);
```

3. Konfigurasi Rate Limit

- 3.1. Buka app/Http/Kernel.php, cari **\$middlewareGroups['api']**

- 3.2. Ubah **'throttle:api'** menjadi **'throttle:60,1'** atau tambahkan middleware custom:

```
protected $middlewareGroups = [
    'api' => ['throttle:60,1',
        \Illuminate\Routing\Middleware\SubstituteBindings::class,
    ];
```

3.3. Atau di routes/api.php:

```
Route::prefix('v1')->middleware([
    'auth:sanctum',
    'throttle:60,1'
])->group(function(){
    // routes...
});
```

4. Debugging

- Tambahkan dd(\$variable) di controller atau service untuk inspect data
- Jalankan request via Postman/Browser dan perhatikan output

5. Tulis Feature Tests

5.1. Buat test file:

```
php artisan make:test ItemApiTest
```

File: tests/Feature/ItemApiTest.php

```
<?php
namespace Tests\Feature;
use Tests\TestCase;
use App\Models\User;
use App\Models\Category;
use Illuminate\Foundation\Testing\RefreshDatabase;
class ItemApiTest extends TestCase {
    use RefreshDatabase;
    protected $user;
    protected $admin;
    protected $category;
    protected function setUp() {
        parent::setUp();
        Category::factory()->create([
            "id" => 1, "name" => "Electronics"
        ]);
        $this->user = User::factory()->create([
            "role" => "user"
        ]);
        $this->admin = User::factory()->create([
            "role" => "admin"
        ]);
    }
    public function test_guest_cannot_access_items(){
        $this->getJson("/api/v1/items")->assertStatus(401);
    }
    public function test_user_can_list_items(){
        $token = $this->user->createToken("api-token")
            ->plainTextToken;
        $this->withHeader("Authorization", "Bearer $token")
            ->getJson("/api/v1/items")
            ->assertStatus(200)
            ->assertJsonStructure([
                "status", "data", "message"
            ]);
    }
}
```

```

    }
    public function test_user_cannot_delete_item(){
        $token = $this->user->createToken("api-token")
            ->plainTextToken;
        $this->deleteJson(
            "/api/v1/items/1",
            [],
            ["Authorization" => "Bearer $token"]
        )->assertStatus(403);
    }
    public function test_admin_can_delete_item(){
        // create item
        $item = \App\Models\Item::factory()
            ->create(["category_id" => 1]);
        $token = $this->admin->createToken("api-token")
            ->plainTextToken;
        $this->deleteJson(
            "/api/v1/items/{$item->id}",
            [],
            ["Authorization" => "Bearer $token"]
        )->assertStatus(204);
    }
}

```

5.2. Siapkan Factories (jika belum):

CategoryFactory & ItemFactory di database/factories

5.3. Jalankan test:

```
php artisan test --filter=ItemApiTest
```

6. Audit Dependency

Di terminal project:

```
composer audit
```

Tinjau daftar vulnerability. Jika ada, perbarui paket:

```
composer update vendor/name
```

Commit perubahan composer.json/composer.lock.

7. Commit & Push

```
git add .
```

```
git commit -m "Add tests, logging, rate limiting & dependency audit"
```

```
git push -u origin feature/testing-debug-security
```

8. Pull Request

8.1. Buka Repository di GitHub

- Pergi ke <https://github.com/USERNAME/inventory>
- Kamu akan melihat notifikasi kuning: "**feature/testing-debug-security had recent pushes**" → klik "**Compare & pull request**"

8.2. Isi Form Pull Request

- Base branch: main (branch tujuan merge)

- Compare branch: feature/auth-sanctum (branch kamu)
- Title: Isi judul PR
- Setelah form diisi, klik tombol "Create pull request"

8.3. Review Kode (Melihat Perubahan)

- Klik tab "Files changed"
- Lihat perubahan kode:
 - Warna hijau = baris kode yang ditambah
 - Warna merah = baris kode yang dihapus

8.4. Merge Pull Request

- Klik tombol "Merge pull request"
- Klik "Confirm merge"